I R I S A
INSTITUT DE RECHERCHE EN INFORMATIQUE ET SYSTÈMES ALÉATOIRES

PUBLICATION
INTERNE
N° 1526

AN ADAPTIVE RIO (A-RIO) QUEUE MANAGEMENT ALGORITHM

JULIO OROZCO, DAVID ROS

ISSN 1166-8687

IRISA
CAMPUS UNIVERSITAIRE DE BEAULIEU - 35042 RENNES CEDEX - FRANCE

INSTITUT DE RECHERCHE EN INFORMATIQUE ET SYSTÈMES ALÉATOIRES
Campus de Beaulieu – 35042 Rennes Cedex – France
Tél. : (33) 02 99 84 71 00 – Fax : (33) 02 99 84 71 71
http://www.irisa.fr

An Adaptive RIO (A-RIO) Queue Management Algorithm

Julio Orozco[*] [**], David Ros[***]

Thème 1 — Réseaux et systèmes
Projet Armor

Publication interne n° 1526 — Avril 2003 — 35 pages

Abstract: In the context of the DiffServ architecture, active queue management (AQM) algorithms are used for the differentiated forwarding of packets. However, correctly setting the parameters of an AQM algorithm may prove difficult and error-prone. Besides, many studies have shown that the performance of AQM mechanisms is very sensitive to network conditions. In this paper we present an adaptive AQM algorithm, which we call *Adaptive RIO* (A-RIO), addressing both of these problems. A-RIO draws directly from the original RIO proposal of Clark and Fang (1998) and the Adaptive RED (A-RED) algorithm described by Floyd et al. (2001). Our simulation results show that A-RIO outperforms RIO in terms of stabilizing the queue occupation (and, hence, queuing delay), while maintaining a high throughput and a good protection of high-priority packets; A-RIO could then be used for building controlled-delay, AF-based services. These results also provide some engineering rules that may be applied to improve the behaviour of the classical, non-adaptive RIO.

Key-words: Active queue management, DiffServ, IP networks, quality of service.

(Résumé : tsvp)

[*] Julio.Orozco@irisa.fr, Julio.Orozco@enst-bretagne.fr
[**] This work was partially supported by the Mexican National Council for Science and Technology (CONACyT), as part of the 26th SFERE/CONACyT joint program.
[***] David.Ros@irisa.fr, David.Ros@enst-bretagne.fr

Centre National de la Recherche Scientifique
(UPRESSA 6074) Université de Rennes 1 – Insa de Rennes

Institut National de Recherche en Informatique
et en Automatique – unité de recherche de Rennes

Un algorithme RIO adaptatif

Résumé : Dans le cadre de l'architecture DiffServ, les algorithmes de gestion active des files d'attente (AQM) sont utilisés pour le relayage différentié des paquets. Cependant, le parametrage d'un algorithme AQM est souvent une tâche délicate. En outre, de nombreuses études ont montré que la performance d'un algorithme AQM est très sensible aux conditions dans le réseau. Dans ce rapport nous présentons un algorithme AQM adaptatif que nous appelons *Adaptive RIO* (A-RIO), lequel permet de pallier aux inconvénients ci-dessus. A-RIO est une extension directe de l'algorithme RIO original, introduit par Clark et Fang (1998), et de l'algorithme RED Adaptatif (A-RED) décrit par Floyd et al. (2001). Les résultats des simulations montrent que la performance d'A-RIO est bien meilleure que celle de RIO, en termes de la stabilité de l'occupation de la file (et donc du délai d'attente), tout en offrant un débit efficace (*throughput*) élevé et une bonne protection aux paquets prioritaires. Ainsi, A-RIO pourrait être employé comme le composant de base d'un service à délai contrôlé basé sur AF. Ces résultats fournissent également des règles d'ingénierie permettant d'améliorer le comportement du mécanisme RIO classique (c.-à-d. non adaptatif).

Mots clés : Gestion active de files d'attente, DiffServ, qualité de service, réseaux IP.

1 Introduction

Active queue management (AQM) is the name given to a type of router mechanisms used in congestion control. AQM mechanisms manage queue lengths by dropping[1] packets when congestion is building up [3], that is, before the queue is full; end-systems can then react to such losses by reducing their packet rate, hence avoiding severe congestion.

Random Early Detection (RED) [11] is one of the first AQM mechanisms to have been proposed, and the one that has been most studied. RED intends to avoid congestion by randomly discarding packets based on the average queue size.

AQM mechanisms are also relevant in the context of DiffServ. The DiffServ architecture has been defined by the IETF to provide IP networks with scalable QoS processing of traffic aggregates, based on a special field in the IP header [23]. The value of this field tells a router what particular treatment, the per-hop behaviour (PHB), should be applied to the packet.

One of the standard PHBs is Assured Forwarding (AF) [12]. AF defines the differentiated forwarding of packets classified in up to four classes. Packets from different classes are processed in different physical queues, managed by a scheduling mechanism. Within each class (or queue), there can be up to three drop precedences. Under congestion, packets marked with the highest priority—that is, the lowest drop precedence—should be the last to be discarded, and vice versa. Such differentiated discarding inside a single queue can be achieved by means of special AQM mechanisms, which are usually extensions of RED.

In this paper we describe an adaptive AQM algorithm, which we call *Adaptive RIO* (A-RIO), suitable for building an AF per-hop behaviour. A-RIO is a straightforward combination of the Adaptive RED (A-RED) algorithm described by Floyd et al. [10] and the RIO algorithm of Clark and Fang [5].

The goal of A-RIO is threefold:

- To simplify the configuration of DiffServ-enabled routers, by alleviating the parameter settings problem most AQM algorithms present.
- To automatically translate a quality-of-service parameter (that is, delay) into a set of router parameters.
- To try to stabilize queue occupation around a target value under heavy network load, irrespective of traffic profile.

[1] Or marking, if a mechanism such as ECN (Explicit Congestion Notification) is used.

This paper is organized as follows. In section 2, we discuss the AQM and DiffServ issues that define the context, motivation and basis of our proposal. In Section 3, the A-RIO algorithm is described in detail. In Section 4, we report on the process and results of a simulation study of A-RIO. Conclusions and perspectives are provided in Section 5.

2 Active Queue Management

To the best of our knowledge, AQM algorithms like RED have not been yet widely implemented in the Internet. One of the drawbacks of RED most frequently mentioned in the literature is the problem of correctly setting its parameters.

As described in [11], the prime goal of RED is to provide congestion avoidance (i.e., a region of low delay and high throughput) by controlling the average queue size at the router. RED calculates the average queue size $\bar{q}$ using an exponential weighted moving average. This average is then compared to two thresholds, min_{th} and max_{th}, with $min_{th} < max_{th}$. If $\bar{q} \leq min_{th}$, no packets are discarded. If $min_{th} < \bar{q} \leq max_{th}$, incoming packets are discarded with a probability p given by a linear function of the average queue size. The maximum discard probability in this region is max_p. In the original algorithm, if the average queue size is higher than the maximum threshold, all packets are discarded. A *gentle* version of RED [8] was proposed later, in which the drop probability p varies from max_p to 1 when the average queue size varies from max_{th} to $2 \cdot max_{th}$. Early discard should signal adaptive sources (like, for instance, TCP sources) to slow down in order to prevent congestion and synchronization. Figure 1 shows the RED mechanism (and its *gentle* version).

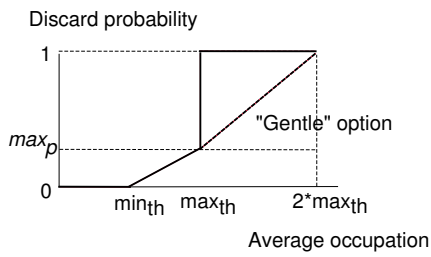

Figure 1: Random Early Discard.

RED has then four user-configurable parameters (the weight w_q used for the averaging function, min_{th}, max_{th} and max_p) whose values can have a large effect on performance. A badly-tuned RED could eventually yield poorer performance than drop-tail, the default discard mechanism in router queues.

Several approaches have been proposed to try to solve the problem of tuning RED's parameters. Among them, we can mention rule-of-thumb recommendations [9], as well as fairly complex analytical models drawing from control theory [14] [26] or queuing theory [2] [22]. The common goal of these models is to be able to calculate the values of RED's parameters based on fixed input data, like the capacity of the bottleneck link, the number of TCP connections, etc.

Works based both on simulation and experimental results ([27], [6], [7], [18], [4] to name but a few) explore the effect of parameter settings on RED's performance. A conclusion is commonly stated: tuning RED is an inexact science and no fixed settings will work well under different congestion scenarios.

2.1 Adaptive RED

A different approach from that of analytical modeling, proposes the on-line adaptation of RED parameters to react to current congestion and traffic patterns.

In [6], a self-configuring RED, called Adaptive RED (A-RED) is proposed. A-RED adapts the maximum discard probability depending on the average queue length $\bar{q}$. The basic assumption behind A-RED is that the value of $\bar{q}$ is a good indicator of the aggressiveness of the mechanism: if the average queue is close to the minimum threshold min_{th}, the discard probability is too aggressive and is leading to under-utilization. On the other hand, if $\bar{q}$ is close to the maximum threshold max_{th} (which implies a higher delay), the discard probability is too conservative. A-RED then adapts the maximum discard probability max_p in order to pull the average queue size to a certain range between thresholds.

In [10], Floyd et al. present a revised version of A-RED[2]. Some significant algorithmic changes are introduced to improve stability and performance. This new version aims at keeping the average queue size in an even smaller interval (q_{low}, q_{high}) between the thresholds, where:

$$q_{low} = min_{th} + 0.4 \cdot (max_{th} - min_{th})$$
$$q_{high} = min_{th} + 0.6 \cdot (max_{th} - min_{th}).$$

[2]In the remainder of this paper, the term A-RED refers exclusively to the revised version of Floyd et al.

Floyd et al. also redesign the parameter setting process to make it more straightforward and clearer from the operator's point of view. The revised A-RED mechanism requires a single input parameter, target delay, which is a direct QoS metric. All fixed parameters, the thresholds and the averaging weight, are then calculated in function of the given target delay. As explained above, the maximum discard probability max_p is the adaptive parameter. It adjusts itself on-line to keep the average queue size within the desired range. Figure 2 shows the pseudo-code for the adaptation function in A-RED.

```
every interval seconds:
      if avg ≥ q_high and max_p ≤ 0.5
            compute increase factor:  α ← min(0.01, max_p/4)
            increase max_p as:  max_p ← max_p + α
      else if avg ≤ q_low and max_p ≥ 0.01
            decrease max_p as:  max_p ← max_p * β

Variables:
      avg:  average queue size
      max_p:  drop probability when avg = max_th

Fixed parameters:
      interval:  0.5 s
      β (decrease factor):  0.9
```

Figure 2: A-RED algorithm.

The salient points of A-RED are the following [10]:

- max_p is adapted between 0.01 and 0.5.
- The lower threshold min_{th} is calculated as a function of the target delay d_t and the link capacity C (in packets per second), with a lower bound of 5 packets. Thus, $min_{th} = \max(5, d_t \cdot C/2)$.
- The upper threshold max_{th} is fixed at $3 \cdot min_{th}$.
- w_q is also calculated in terms of the link bandwidth as $w_q = 1 - \exp(-1/C)$.
- A-RED uses the *gentle* variant of RED.

- The adaptation function of max_p uses an AIMD rule (*Additive Increase Multiplicative Decrease*). This rule aims at avoiding brutal changes in max_p that could lead to heavy oscillations of the queue size.
- If the load changes abruptly, the average queue size could find itself out of the target interval. The increase/decrease factors α and β are fixed so that average queue size regains the target interval in no more than 25 seconds.

Based on the performance evaluation results presented in [10], it may be concluded that A-RED might be beneficial for network operators, because it seems to alleviate the problem of tuning parameters while providing a more predictable delay.

2.2 Active Queue Management in DiffServ Networks

As mentioned in Section 1, AQM mechanisms are also a key building block of DiffServ networks, particularly for the AF PHB. Nevertheless, the goals of AQM in DiffServ are somewhat different from those in a best-effort context. While the objective of AQM mechanisms in best-effort networks is congestion avoidance, in a DiffServ environment they are used mainly for prioritized discard.

In an AF-enabled router, packets of different priorities are forwarded through the same physical queue. Under congestion, packets of the highest priority should be the last to be discarded. In AF, the term *precedence* is used to define the different discard priorities. Packets of the highest priority are those with the lowest precedence, and vice versa.

RED with In and Out (RIO), proposed by Clark and Fang in [5], is the basic AQM mechanism suitable for the setup of the AF per-hop behaviour. It could be said that [5] contributed greatly to the conception of the AF PHB itself. The authors propose an *allocated capacity* framework in which users get an assured bandwidth even under congestion, with the possibility of being granted a higher rate if the network is not congested. The framework is composed of two elements: a marker for the network's edges and a differential discard mechanism for the backbone routers. The marker meters input flows and tags packets as *In* (in-profile) if they conform to the assured rate or *Out* (out-of-profile) if they do not. The differential dropping algorithm for backbone routers is called RED with In and Out (RIO). RIO is a direct extension of RED that uses two sets of parameters to differentiate the discard of *In* and *Out* packets. For deciding whether to discard *Out* packets, RIO uses the average size of the total queue, formed by *In* and *Out* packets. For *In* packets, it uses the average size of a virtual queue formed only by *In* packets. Figure 3 shows the pseudo-code for the RIO algorithm.

```
for every incoming packet,
    if it's an In packet
          update avg_in, the average queue size for In packets
    update avg_total, the average queue size for In plus Out packets

if it's an In packet
    if min_in < avg_in ≤ max_in
          compute probability p_in
          with probability p_in, discard this packet
    else if max_in < avg_in
          discard this packet

if it's an Out packet
    if min_out < avg_total ≤ max_out
          compute probability p_out
          with probability p_out, discard this packet
    else if max_out < avg_total
          discard this packet
```

Figure 3: RIO algorithm.

RIO has been generalised to handle $n > 2$ precedences following the same principles. The discard probability for packets of precedence $1 \leq j < n$ depends on the average size $avg^{(j)}$ of a virtual queue containing only the packets with precedences 1 to j. For packets with precedence n (i.e., the lowest priority), the discard probability is a function of the average occupation of the "physical" (total) queue. This original method was eventually called RIO-C (*Coupled*) [25] to differentiate it from methods proposed later. For example, Weighted RED (WRED) [1] uses the total average queue size for all precedences, whereas RIO-DC (*Decoupled*) [25] calculates the drop probability for packets of precedence j as a function of the average number of packets of the same precedence. Figure 4 shows a diagram of a RIO-C version for $n = 3$ precedences. In this case, colors are associated to packet precedences: green for the lowest (highest priority), yellow for the middle and red for the highest.

RIO-C discriminates packets of different precedences in three ways. The first one is the use of different thresholds for different precedences, so that discard begins "earlier" for packets of higher precedences. The second way is the use of drop probabilities that increase at different rates for different priorities. The third way

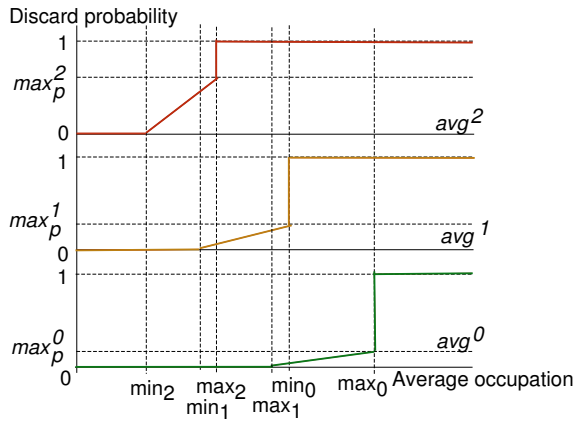

Figure 4: RED with In and Out (RIO), for the three-precedence case.

lies in the coupled calculation of the discard probability; the fact that the discard probability for precedence j uses the average number of packets of *all* lower precedences yields a significant discrimination. Note that the first two ways of achieving differentiation are based simply on different parameter settings, and that they are not mutually exclusive. This gives a broad range of possible settings, which may be roughly grouped in three categories: same thresholds/different drop probabilities, different thresholds/same drop probabilities or different thresholds/different probabilities.

As we mentioned before, there are no precise rules for tuning RED's four parameters. This problem gets magnified with RIO: for a n-precedence RIO, in principle $3n + 1$ parameters should be set; that is, $2n$ thresholds and n maximum drop probabilities $max_p^{(i)}$, $i = 1, \ldots, n$, plus the weight w_q used for averaging queue occupation—assuming the same w_q is used for all virtual queues. The problem of parameter setting obviously becomes more complex and, therefore, has also become a subject of research.

As the basis for an extension, the work in [24] presents the results of a performance evaluation of RIO. The study was done by simulation and provides with some guidelines for parameter setting. A similar approach can be found in [15]. An experimental study is described in [16], where RIO performance is evaluated for different parameter settings and traffic patterns. The study focuses on throughput and the level of protection offered to high priority packets.

May et al. [19] and Malouch and Liu [17] follow an analytical approach. In the former, the authors present a markovian model of a RIO router. The latter paper describes a set of equations that represents TCP sources and a RIO router; the equations are numerically solved with the fixed-point method.

In general, all these studies illustrate the difficulty of tuning RIO in order to achieve a predictable performance. This issue is very relevant, since a key idea behind DiffServ is to allow the provisioning of enhanced services. The operator should know how to set up services with some (loose) guarantees in rate or delay.

3 An Adaptive RIO (A-RIO) Algorithm

We will now describe our proposal for an AQM mechanism called Adaptive RIO (A-RIO). A-RIO is a direct extension of both the revised A-RED (section 2.1) and RIO-C algorithms (section 2.2). A-RIO follows the approach of the former, performing an on-line automatic adaptation of the mechanism to get a more predictable performance, together with a more straightforward parameter tuning. Several approaches have been proposed in the literature for dealing with the tuning of RED; we have chosen A-RED because of its simplicity (both in concept and in implementation).

3.1 Motivation

With this proposal, we look forward to alleviate the problem of parameter tuning in the context of DiffServ AF networks. Like A-RED, A-RIO needs a single input parameter, the *target delay*, which is translated into the required set of router parameters. This feature could be very interesting for providers of differentiated services: configuring routers in terms of delay—a QoS metric directly related to service specifications and customer requirements—should be much easier than in terms of more "abstract" parameters like queue thresholds, discard probabilities or averaging weights.

From a performance point of view, A-RIO seeks to achieve a maximum throughput while keeping delay in a bounded, predictable interval when the queue load is high. In the context of DiffServ, we also intend to assure the correct discrimination of packets marked with different precedences.

3.2 Definition

The A-RIO algorithm is based on two main principles. The first one is the use of *a full Adaptive RED instance for each precedence level* in the AF class (physical queue). The second principle is the use of *completely overlapped thresholds for all precedences.* The pseudo-code of A-RIO is shown in Fig. 5 while the concept, for a three-precedence queue, is depicted in Fig. 6.

The following characteristics of A-RED are kept unchanged in A-RIO:

- The adaptive parameters, $max_p^{(i)}$, are varied between 0.01 and 0.5.
- The lower threshold min_{th} is calculated as: $min_{th} = \max(5, d_t \cdot C/2)$, where d_t denotes the target delay (in s) and C the link capacity (in packets/s). The upper threshold max_{th} is given by: $max_{th} = 3 \cdot min_{th}$.
- w_q is also calculated in terms of the link bandwidth as $w_q = 1 - \exp(-1/C)$.
- The *gentle* variant [8] of RED is used throughout. This corresponds, in Fig. 6, to the interval $max_{th} \leq avg^{(i)} \leq 2 \cdot max_{th}$.
- The adaptation function of the $max_p^{(i)}$ uses an AIMD rule.
- The increase/decrease factors α and β follow the same guidelines as in A-RED.

Design decisions for A-RIO have been made with a guideline in mind: the target delay, which should prevail for every traffic mix *as long as the load is non negligible.* The goal of the algorithm is to keep average queue size in the interval (q_{low}, q_{high}), where $q_{low} = min_{th} + 0.4 \cdot (max_{th} - min_{th})$ and $q_{high} = min_{th} + 0.6 \cdot (max_{th} - min_{th})$.

To better explain this, let us discuss the scenario of a RIO-C queue with non-overlapping thresholds. The precedences are 1 (*In*) and 2 (*Out*). If the total *In* rate is low compared to the link capacity, under a heavy load the *Out* thresholds and discard probability will be active, (hopefully) keeping the average queue size between min_2 and max_2. However, if most of the traffic is *In*, the average queue size will be somewhere between min_1 and max_1. Thus, staggered thresholds may yield different average queue sizes for different traffic mixes. This is a drawback if we want a predictable delay for the queue as a whole and under any congestion scenario. Therefore, we propose the use of the same thresholds for all precedences. This way, the adaptation mechanism of A-RED should pull the average queue size to the same bounded interval, regardless of the traffic mix.

```
for every incoming packet of drop precedence i,
    for every drop precedence j = i, i+1, ..., n
          update avg^(j) as:  avg^(j) ← avg^(j) · (1 − w_q) + q^(j) · w_q
          every interval time units update max_p^(j):
              if avg^(j) > q_high and max_p^(j) < 0.5
                   compute increase factor:  α ← min(0.01, max_p^(j)/4)
                   increase max_p^(j) as:  max_p^(j) ← max_p^(j) + α
                   if j < n then:  max_p^(j) ← min(max_p^(j), max_p^(j+1))
              else if avg^(j) < q_low and max_p^(j) > 0.01
                   decrease max_p^(j) as:  max_p^(j) ← max_p^(j) * β
                   if j > 0 then:  max_p^(j) ← max(max_p^(j), max_p^(j−1))
    if min_th < avg^(i) ≤ max_th
          calculate p^(i) as in A-RED
          discard this packet with probability p^(i)
    else if max_th < avg^(i) ≤ 2 * max_th
          calculate p_gentle^(i) as in A-RED
          discard this packet with probability p_gentle^(i)
    else if avg^(i) > 2 * max_th
          discard this packet

Variables:
    avg^(i):     average queue size for precedence i (coupled:
                 counts number of packets with precedence from 1 to i)
    max_p^(i):   drop probability for precedence i when avg^(i) = max_th
    p^(i):       discard probability for precedence i
    p_gentle^(i): discard probability for precedence i in gentle zone

Fixed parameters:
    interval:  0.5 s;
    β (decrease factor):  0.9
```

Figure 5: A-RIO algorithm.

However, the use of overlapped thresholds has implications in differentiation. As stated in Section 2.2, RIO-C differentiates discard in three basic ways: different thresholds, different discard functions and coupled virtual queues. In A-RIO, with the choice of overlapped thresholds, we have excluded the first method. On the

other hand, A-RED is based on adapting max_p, and using different $max_p^{(i)}$'s are another method of discrimination. Note that the adaptation algorithm is such that: $max_p^{(i)} \leq max_p^{(i+1)}, \forall i \in \{1, \ldots, n-1\}$; together with the coupled virtual queues, this restriction should provide with enough assurance of discrimination.

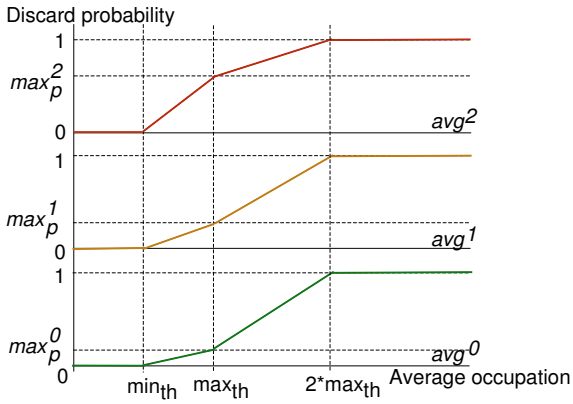

Figure 6: A-RIO for three precedences.

Concerning implementation issues, note that the complexity of A-RIO is roughly equivalent to that of A-RED and RIO-C combined. Moreover, remark that A-RIO does not store or compute per-flow information, so scalability is (in principle) not an issue.

4 Performance Evaluation of A-RIO

4.1 Overview

In order to evaluate the performance of the proposed A-RIO algorithm, an extensive series of simulations was carried out using the ns-2 network simulator. The code[3] for the algorithm is based on the DiffServ module included in the 2.1b9a release of ns-2.

The main goal of the experiments is to verify that A-RIO effectively keeps the average queue size of an AF router at the desired interval under a variety of traffic

[3]Both the code and the simulation scripts can be downloaded from `http://www.rennes.enst-bretagne.fr/~jorozco/aqm.htm`.

types and loads. A-RIO should accomplish this goal while providing with protection of high priority packets and fairness of bandwidth allocation.

We checked A-RIO's performance by comparing it to that of the original, static RIO Coupled and of a modified version that we call Gentle-RIO or G-RIO. As its name suggests, G-RIO implements the *gentle* mode of RED. Additionally, the values for G-RIO's parameters (thresholds and averaging weights) are set using *the same rules* originally proposed for A-RED and adapted to A-RIO. In other words, G-RIO corresponds to an A-RIO *without* on-line adaptation of the $max_p^{(i)}$.

The three AQM mechanisms (A-RIO, RIO, G-RIO), several traffic types and marking profiles were tested; a total of 30 scenarios per AQM algorithm were used. The details are presented in the following sections.

4.2 Simulation configuration

Figure 7 shows the simplest topology used for the simulations. An AF dumbbell backbone is set up with two routers, r0 and r1. Test traffic is generated by 100 individual TCP sources (src0–src99) connected to ingress router r0 via links of 1 Mb/s each. Each one of the source nodes is also an edge router where traffic is marked by means of a trTCM (Two-Rate Three Color Marker) [13]. All traffic is destined to a single sink node (sink 0) and is marked to be forwarded in a single AF class (physical queue) in the network's core. The backbone link has a rate of 30 Mb/s (i.e., 30% of aggregated access capacity). The AQM algorithm of interest in each scenario is activated in a single physical queue at the backbone output interface of the router r0.

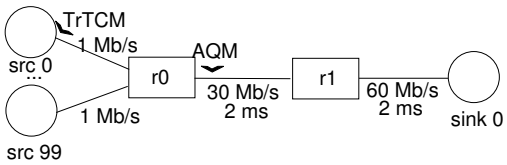

Figure 7: Dumbbell simulation topology.

4.3 Traffic sources

Table 1 summarizes the test scenarios. Simulations were grouped in six "cases", depending on the type of input traffic and round-trip time (RTT) of each source; in

other words, each case corresponds to a given traffic profile. The number of TCP sources remains constant at 100.

Case	AQM	No. of Sources			RTTs	Assured Rates
		Total	FTP	Pareto On/Off		(% of link capacity)
1	A-RIO, RIO, G-RIO	100	100	0	$=$	25, 50, 75, 100, 125
2	A-RIO, RIO, G-RIO	100	100	0	$\neq$	25, 50, 75, 100, 125
3	A-RIO, RIO, G-RIO	100	0	100	$=$	25, 50, 75, 100, 125
4	A-RIO, RIO, G-RIO	100	0	100	$\neq$	25, 50, 75, 100, 125
5	A-RIO, RIO, G-RIO	100	20	80	$=$	25, 50, 75, 100, 125
6	A-RIO, RIO, G-RIO	100	20	80	$\neq$	25, 50, 75, 100, 125

Table 1: Simulation cases.

In cases 1 and 2, traffic is generated by 100 long-lived FTP sources, that is, sources having an infinite amount of data to send[4]. In cases 3 and 4 traffic consists of Pareto On/Off data sources over TCP, while traffic for cases 5 and 6 is a combination of 80 Pareto On/Off and 20 FTP sources. RTTs of access links are the same (50 ms) for all TCP connections in cases 1, 3 and 5, and different (going from 40 to 535 ms in steps of 5 ms) in cases 2, 4 and 6. Hence, the complexity of the traffic mix increases from pure long-lived FTP sources and equal RTTs in case 1 to mixed FTP and On/Off with heterogeneous RTTs in case 6. Simulations were run for 120 s; sources start at random times between $t = 2$ and $t = 12$ s. Table 2 summarises the parameter values for the traffic sources.

Traffic type	Parameter	Value
FTP	Transport protocol	TCP Reno
	Receiver window	15 packets
On/Off	Transport protocol	TCP Reno
	Receiver window size	15 packets
	Burst and idle time distribution	Pareto
	Mean burst time	350 ms
	Mean idle time	650 ms
	Peak rate	1000 kb/s
	Shape (for the Pareto distribution)	1.5

Table 2: Parameters of traffic sources.

[4]In the remainder of this paper, we will refer to this kind of source simply as "FTP".

4.4 Marking

For each one of the six cases, five different marking profiles (all based on a three-color marker) were tested. The marking schemes were defined by the total assured rate as a percentage of bottleneck link capacity. For example, a marking scheme of 25% means that the total assured rate is 25% of the bottleneck's bandwidth; for the bottleneck bandwidth of 30 Mb/s, a total assured rate of 25% equals 7.5 Mb/s. This rate is then divided between the 100 individual sources. As shown in Table 1, assured rates of 25%, 50%, 75%, 100% and 125% were tested, in order to evaluate the performance of the AQM at different levels of subscription.

The trTCM is made of two successive token buckets and has four parameters: committed information rate (CIR), committed burst size (CBS), peak information rate (PIR) and peak burst size (PBS). Given the assured (or committed) rate CIR, the remaining parameters of each trTCM were set up using the engineering rules proposed in [21]. These rules establish the following proportional relationships:

$$\text{PIR} = \alpha \text{ CIR}$$
$$\text{CBS} = \beta \text{ CIR}$$
$$\text{PBS} = \beta \text{ PIR}$$

with $\alpha = 2$ and $\beta = 1.5$.

4.5 AQM parameters

For each one of the 30 scenarios (i.e., six cases of five marking profiles each), a simulation is run for each one of the three chosen AQM algorithms: A-RIO, G-RIO and RIO. As seen in section 3, a single parameter is required for configuring an A-RIO queue: the target delay. Values for the thresholds and averaging weight w_q are automatically calculated based on that delay. These values are then manually assigned to parameters for the simulations with G-RIO.

In the case of RIO, a model of partially-overlapped thresholds is used. This model is taken from [20] and sets thresholds for the three precedences in terms of the queue size (Table 3). Remark that similar settings can be found in [24] and [15].

Table 4 shows the AQM parameter values used for the simulations. Values for A-RIO and G-RIO are calculated for a target delay of 50 ms, a bandwidth of 30 Mb/s and a packet size of 1000 bytes. All maximum discard probabilities are set at the same values; in the case of A-RIO, the values of the $max_p^{(i)}$ are of course initialisation values, since they change throughout the simulation. The size of the physical queue is 1000 packets.

Color	th_{min}	th_{max}
Red	$0.10Q$	$0.25Q$
Yellow	$0.20Q$	$0.40Q$
Green	$0.35Q$	$0.65Q$

Q: Queue size

Table 3: RIO model for computing threshold values.

Precedence	"Color"	A-RIO/G-RIO				RIO			
		th_{min}	th_{max}	max_p	w_q	th_{min}	th_{max}	max_p	w_q
2	Red	94	281	0.2	0.0003	100	250	0.2	0.002
1	Yellow	94	281	0.1	0.0003	200	400	0.1	0.002
0	Green	94	281	0.02	0.0003	350	600	0.02	0.002

Table 4: AQM parameters.

4.6 Results

4.6.1 Average queue sizes

The plots in Fig. 8 summarise the results of average queue sizes per AQM for the 30 scenarios (results for a given traffic profile are shown. Averages are calculated after a warm-up time of 20 s, when all sources are active.

According to the thresholds shown in Table 4, the target average queue size for A-RIO should be around 187.5 packets. It can be seen that A-RIO effectively keeps the queue size close to the target for a wide variety of traffic loads and marking profiles. Only in case number 4 is the average occupation lower for all assured rates. This is due to the fact that the queue remains lightly loaded (because of the traffic characteristics), at a level which is below A-RIO's minimum threshold for adaptation. We could say that, in such a case, A-RIO (like A-RED), lets things follow their natural course. More details of this scenario are described in section 4.7.3.

By contrast, the average queue size for RIO varies in a much wider interval for the different scenarios[5]. This means that delay could vary in a fairly broad interval, making it difficult for a service provider to offer a service with consistent (upper) delay bounds. Finally, note that the results with G-RIO are not as consistent as those with A-RIO, but the average queue size varies in a smaller interval when compared to RIO. Hence, *even without adaptation*, the A-RED rules for setting

[5]Note that, in some cases, RIO performs "better" than A-RIO, in the sense that a lower delay is obtained. However, what we are interested in is *consistent* performance under a wide variety of loads and traffic mixes, something that plain RIO does not seem able to achieve.

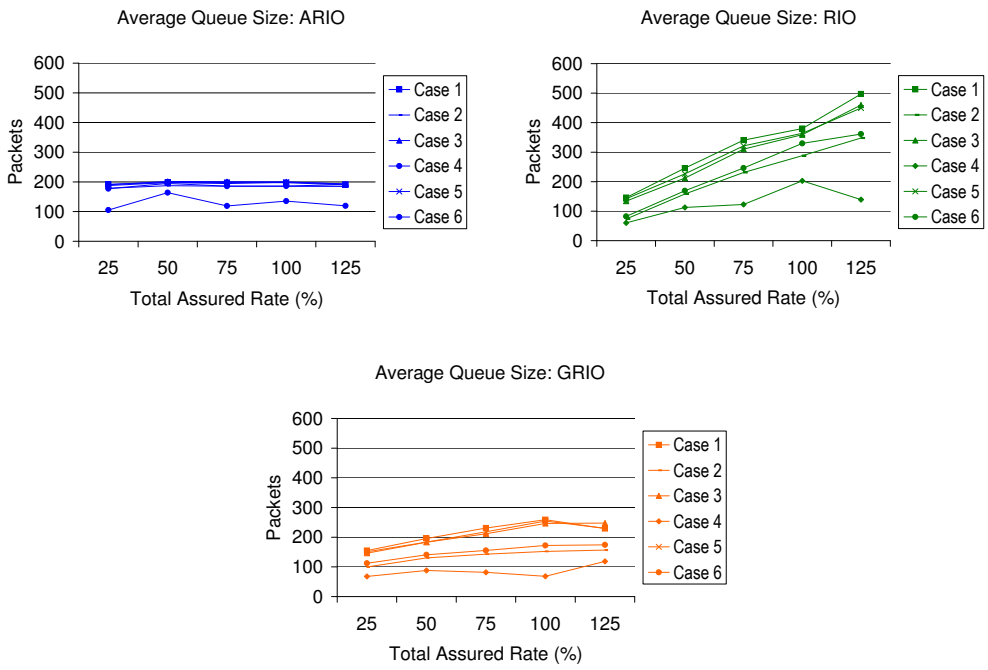

Figure 8: Average queue sizes per AQM, case and marking profile

parameters and the use of overlapped thresholds can be used in static RIO to obtain a more predictable performance in terms of delay.

We mentioned previously that A-RIO should achieve its main goal (keeping average queue close to a target size) while still providing protection for priority packets and fairness of bandwidth allocation. The results for the metrics of these requirements are presented in the following sections.

4.6.2 Protection of priority packets

Table 5 shows the results for utilization and protection of priority packets in the bottleneck link. The table includes, for each AQM mechanism, the mean overall link utilization (total throughput/capacity) and the percentage of discarded green packets. Note that utilization is fairly the same for every case and AQM.

Percentages of discarded green packets are slightly higher for A-RIO and G-RIO than for RIO. This was expected, due to the fact that the former two use overlapped thresholds. Nevertheless, these percentages are reasonably low (the overall mean of discarded green packets is 0.51%, both for A-RIO and G-RIO); besides, remark that green packet losses are $> 1\%$ only when the assured rate is either 100% or 125% of the link capacity (i.e., under over-subscription scenarios).

Case	Assured Rate	Utilization (%)			Discarded Green packets (%)		
		RIO	A-RIO	G-RIO	RIO	A-RIO	G-RIO
1	25	92.84	92.98	93.29	0.00	0.00	0.00
	50	93.22	93.35	92.85	0.00	0.05	0.06
	75	93.44	92.87	93.17	0.00	0.31	0.79
	100	93.30	93.62	93.16	0.19	1.74	1.51
	125	93.39	93.05	92.51	1.13	2.77	2.56
2	25	89.34	91.85	87.35	0.00	0.00	0.00
	50	91.47	90.98	91.15	0.00	0.01	0.00
	75	92.07	91.17	91.91	0.00	0.20	0.11
	100	91.61	91.35	92.43	0.00	0.33	0.36
	125	91.80	92.03	91.64	0.06	0.44	0.49
3	25	90.56	89.97	91.12	0.00	0.00	0.00
	50	91.12	90.55	91.26	0.00	0.02	0.02
	75	91.30	90.11	91.11	0.00	0.29	0.53
	100	91.24	90.02	90.56	0.06	1.18	1.29
	125	90.99	89.86	89.77	0.79	2.51	1.82
4	25	87.66	89.05	87.77	0.00	0.00	0.00
	50	90.20	89.83	89.21	0.00	0.00	0.00
	75	90.15	89.68	89.31	0.00	0.03	0.00
	100	90.36	89.43	89.18	0.00	0.12	0.01
	125	89.27	89.63	90.01	0.00	0.12	0.20
5	25	88.79	91.58	91.58	0.00	0.00	0.00
	50	91.38	92.23	91.71	0.00	0.01	0.01
	75	92.18	91.72	92.50	0.00	0.23	0.63
	100	92.61	92.30	91.62	0.11	1.21	1.37
	125	92.00	92.38	90.59	0.73	2.52	2.19
6	25	88.79	91.45	88.60	0.00	0.00	0.00
	50	91.31	91.61	91.29	0.00	0.01	0.00
	75	91.63	91.99	91.50	0.00	0.18	0.15
	100	91.37	91.69	91.35	0.00	0.34	0.51
	125	91.62	91.48	90.96	0.13	0.69	0.66
Mean		91.23	91.33	91.02	0.11	0.51	0.51

Table 5: Percentages of overall link utilization and discarded green packets.

4.6.3 Fairness

Given the fact that all 100 sources are marked for the same individual assured rate, all of them should get the same bandwidth in all scenarios, at least in theory. It

is expected, though, that in the cases with heterogeneous RTTs, bandwidth will be allocated unevenly. Under these considerations, Table 6 allows us to evaluate the degree of fairness in each scenario. The table presents the coefficient of variation (standard deviation/mean) for the observed per-flow throughput. These results show that, in general, dispersion is higher when RTTs are different (cases 2, 4 and 6), but there seems to be no major differences between RIO, A-RIO and G-RIO.

Case	Assured Rate (%)	RIO	A-RIO	G-RIO	Case	Assured Rate (%)	RIO	A-RIO	G-RIO
1	25	0.0485	0.0527	0.0495	2	25	0.3964	0.3852	0.3967
	50	0.0144	0.0276	0.0336		50	0.1646	0.2384	0.2966
	75	0.0304	0.0364	0.0397		75	0.2408	0.3205	0.2823
	100	0.0220	0.0343	0.0366		100	0.2078	0.3313	0.3168
	125	0.0489	0.0697	0.0847		125	0.2405	0.3571	0.3571
3	25	0.1393	0.1258	0.1551	4	25	0.2364	0.3095	0.2442
	50	0.1812	0.1316	0.1223		50	0.2160	0.2179	0.2337
	75	0.1964	0.1623	0.1579		75	0.2386	0.2556	0.2574
	100	0.1530	0.0770	0.1244		100	0.2508	0.2867	0.2715
	125	0.1336	0.1175	0.1735		125	0.2523	0.2493	0.2805
5	25	0.1150	0.1845	0.1314	6	25	0.3640	0.3041	0.3380
	50	0.1129	0.1316	0.1326		50	0.1886	0.1777	0.2301
	75	0.1280	0.1568	0.1096		75	0.2222	0.2774	0.2462
	100	0.0836	0.1018	0.0994		100	0.1493	0.2448	0.2590
	125	0.1485	0.1287	0.1467		125	0.2205	0.2874	0.3187

Table 6: Fairness measure (coefficient of variation of per-flow throughput).

4.7 Detailed results of selected scenarios

In this section we present detailed results for three scenarios that illustrate A-RIO's adaptive behavior under different traffic conditions.

4.7.1 Upward adaptation

The first scenario is case 3 (On/off traffic, homogeneous RTTs) with a total assured rate of 100%. Figure 9 shows three plots per AQM (A-RIO and RIO). The top row shows the evolution in time of the maximum discard probabilities $max_p^{(i)}$, for each precedence level. Plots in middle row show the average occupation of each virtual queue. Since averages are coupled (Section 2.2), the average queue size for precedence 2 counts packets of all precedences. Finally, plots in bottom row illustrate the total throughput and total average queue size.

The traffic mix in this scenario is mainly composed of green packets (100% assured rate). Thus, the total average occupation (red, or precedence 2) is almost the same as the "green" occupation. With A-RIO, the discard probability of all precedences increases after $t = 20$ s. The increase is rather sharp in the case of yellow (1) and red (2) precedences. This increase is due to the fact that, according to the basic assumptions behind A-RIO, the mechanism needs to be more aggressive to keep occupation near the target of 187.5 packets. Plots in third row show how A-RIO succeeds in keeping average occupation close to the target. As shown in table 7, the observed average occupation is 200.1 packets. With RIO, average occupation is considerably higher (359.1 packets).

The gains of A-RIO in terms of average occupation (and, hence, delay) are obtained without affecting other performance criteria. Table 7 shows protection and fairness data for this scenario. Protection is measured by the percentage of discarded green packets. For A-RIO, this percentage is 1.18% (higher than the 0.06% recorded for RIO, but this is to be expected due to the overlapped thresholds). As for fairness, since all individual flows are marked with the same assured rate, they should get an equal share of the available capacity (0.3 Mb/s per flow). Table 7 shows the mean per-flow throughput and its corresponding coefficient of variation (CV). This measure of dispersion indicates how much individual throughputs vary around the mean. It can be seen that mean per-flow throughput is the same for A-RIO and RIO, but the CV is lower in the case of A-RIO.

Measure	A-RIO	RIO
Average queue size	200.14	359.09
Protection		
Discarded green packets (%)	1.19	0.06
Fairness		
Mean per-flow throughput (Mb/s)	0.3	0.3
CV	0.08	0.15

Table 7: Case 3-100%: results.

4.7.2 Downward adaptation

The scenario analysed in this section is case 6 (80 On/Off and 20 FTP flows, heterogeneous RTTs) with a total assured rate of 75%. Here, the traffic mix is composed not only of green packets, but also of about 25% yellow packets. Plots and results, as described in the previous section, are shown in Figure 10 and Table 8. In this

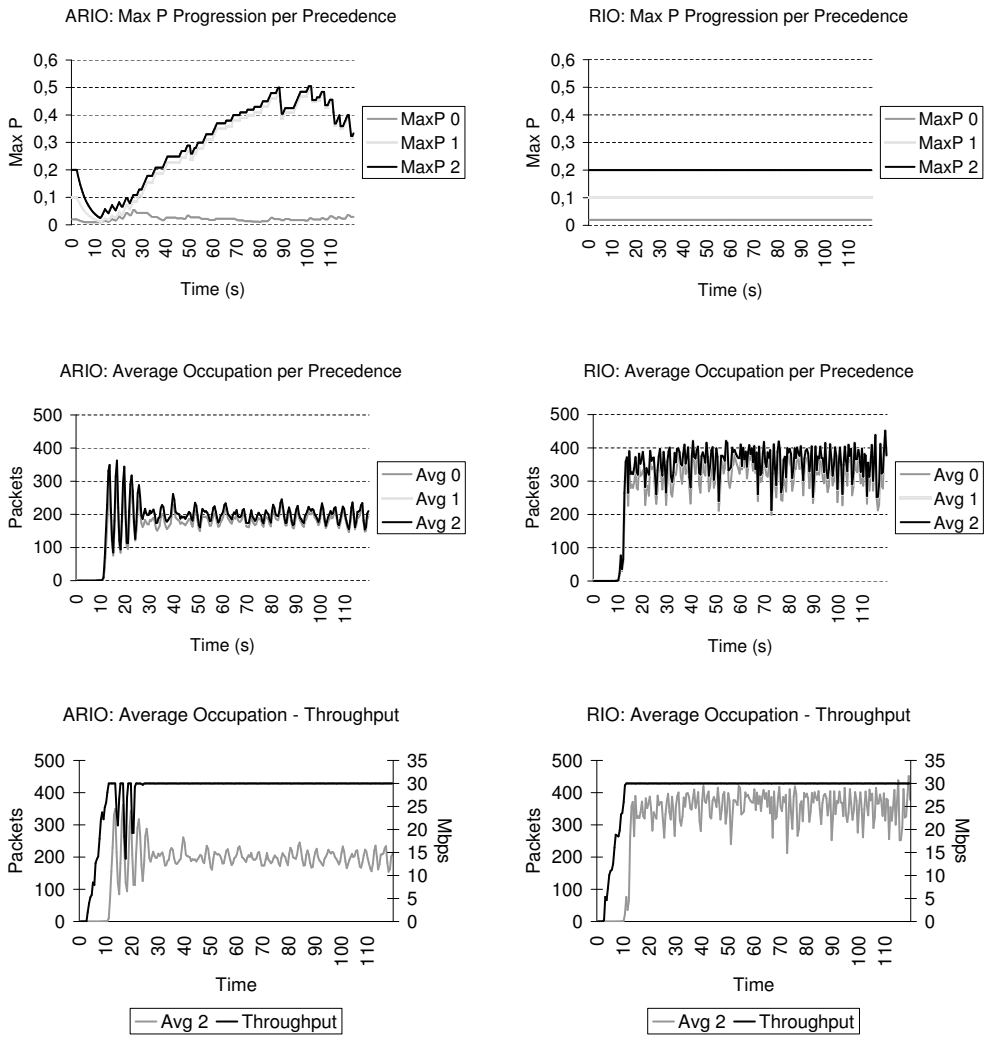

Figure 9: Case 3-100%: plots.

scenario, with A-RIO, all maximum discard probabilities decrease from their initial values to maintain an average queue size of 185.8 packets, very close to the target. Since the assured rate is lower than in the previous example, less green packets are

discarded. Finally, even if mean per-flow throughput is the same as in the previous example, heterogeneous RTTs cause more unfairness in bandwidth sharing.

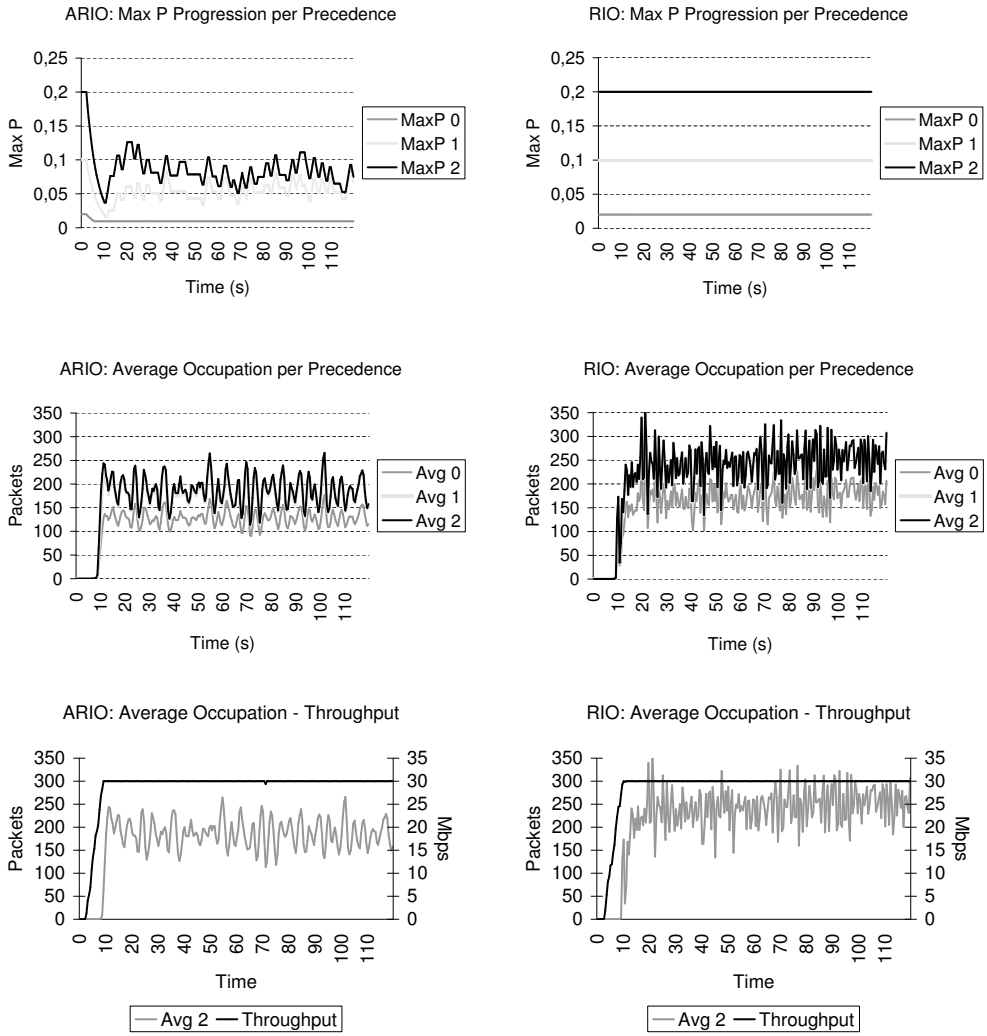

Figure 10: Case 6-75%: plots.

Measure	A-RIO	RIO
Average queue size	185.82	246.09
Protection		
Discarded green packets (%)	0.18	0.0
Fairness		
Mean per-flow throughput (Mb/s)	0.3	0.3
CV	0.28	0.22

Table 8: Case 6-75%: results.

4.7.3 Lightly occupied queue

This section presents one of the scenarios of case 4 (pure On/Off traffic, heterogeneous RTTs). This particular type of traffic keeps the bottleneck queue lightly occupied, a behaviour observed with all marking profiles. The plots and results for a total assured rate of 75% are shown in Figure 11. It can be seen that A-RIO interprets low queue size as the mechanism being too aggressive. A-RIO then decreases all the $max_p^{(i)}$ to their minimum value. Nevertheless, the small queue size is due to traffic characteristics, not to the discard action being aggressive. Queue occupation stays persistently low and, as stated in section 4.6.1, A-RIO lets things follow their natural course. Table 9 presents the corresponding data for average queue size, protection of priority packets and fairness. Average size is significantly lower than target, virtually no green packets are lost and fairness is comparable for both A-RIO and RIO.

Measure	A-RIO	RIO
Average queue size	118.88	122.78
Protection		
Discarded green packets (%)	0.03	0.0
Fairness		
Mean per-flow throughput (Mb/s)	0.3	0.3
CV	0.26	0.24

Table 9: Case 4-75%: results.

4.8 Effect of the number of TCP connections

Using the same basic dumbbell topology, another set of tests was carried out to test the robustness of A-RIO with respect to the number of TCP connections. For

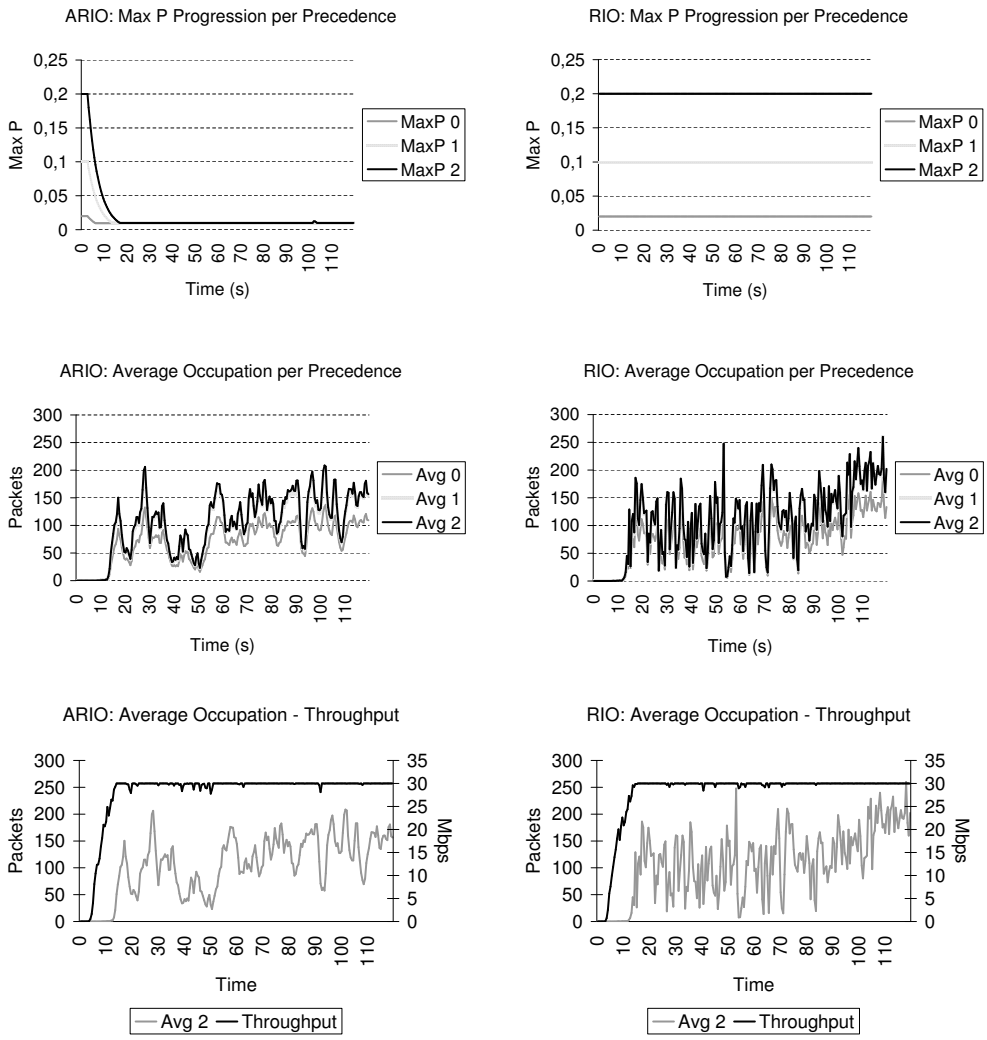

Figure 11: Case 4-75%: plots.

this purpose, a single traffic mix and a single marking profile were kept, while we varied the number of sources (25, 50, 100, 150 and 200). The traffic mix tested was that of the case 6 (80 On/Off sources, 20 FTP sources, mixed RTTs), with a total assured rate of 75% of the bottleneck's capacity. Figure 12 show the plots for three

scenarios (25, 100 and 200 connections). The plots at left illustrate the evolution of discard priorities, while plots at right show the total throughput and the average queue size. The plots are complemented by data in Table 10, where the target and observed average queue sizes, together with average $max_p^{(2)}$, are presented for each scenario. The main point of interest here is the difference in the progression of discard probabilities. Due to the topology used, for the same target delay, increasing the number of connections requires the increase of the bottleneck's bandwidth (this explains why the target queue size in Table 10 also varies with the number of sources). Thus, for a higher number of connections, the mechanism has to keep a bounded occupation in a faster link. This means that it has to be less aggressive and decrease the discard probability. As seen in the table, A-RIO succeeds in keeping average queue size near the target size in all cases, while the decreasing trend of average $max_p^{(2)}$ illustrates the required decreasing aggressiveness.

Number of TCP connections	Average queue size		Average $max_p^{(2)}$
	Target	Observed	
25	46.5	49.37	0.1441
50	94	94.86	0.1054
100	187.5	185.82	0.0829
150	281.5	280.82	0.0880
200	375.5	368.82	0.0216

Table 10: Effect of the number of connections—target and measured average queue sizes.

4.9 A more complex topology

4.9.1 Description

The performance of A-RIO was also simulated using a more complex topology with multiple bottlenecks. Figure 13 shows this topology. It's an AF backbone made of six routers. As in the dumbbell topology, the flows of interest are generated by 100 nodes (src0 to src99) connected to the router rtr0 via access links of 1 Mb/s. All links between backbone routers are of 30 Mb/s. Background traffic aggregates are generated by other nodes connected to the backbone routers (aux0 to aux7) via 30 Mb/s links. The goal is to have aggregates that traverse one or two backbone routers. These aggregates are depicted by dotted arrows.

For this topology, most of the conditions used for the dumbbell topology were kept:

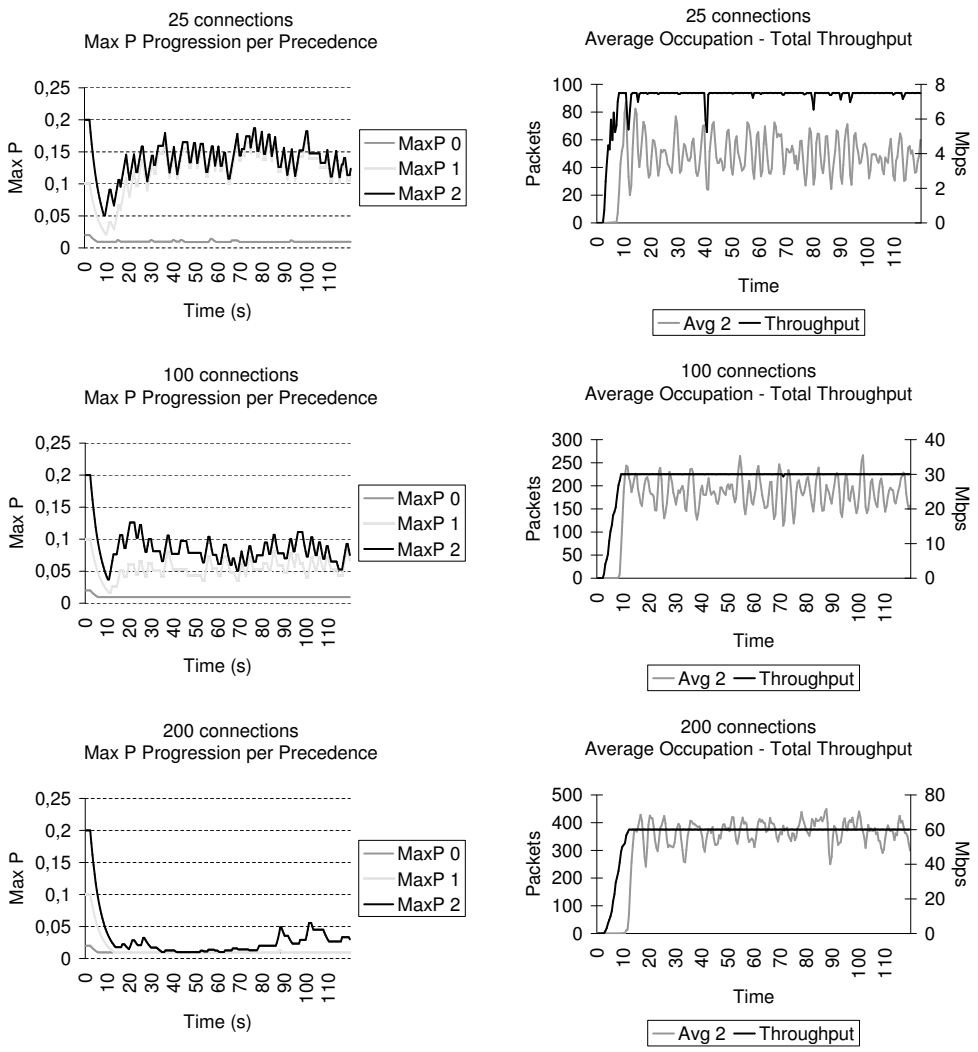

Figure 12: Effect of the number of connections.

- The AQM parameters (Table 4).
- The six cases (i.e., traffic profiles) and five marking profiles (Table 1).
- Characteristics of traffic sources (Table 2).

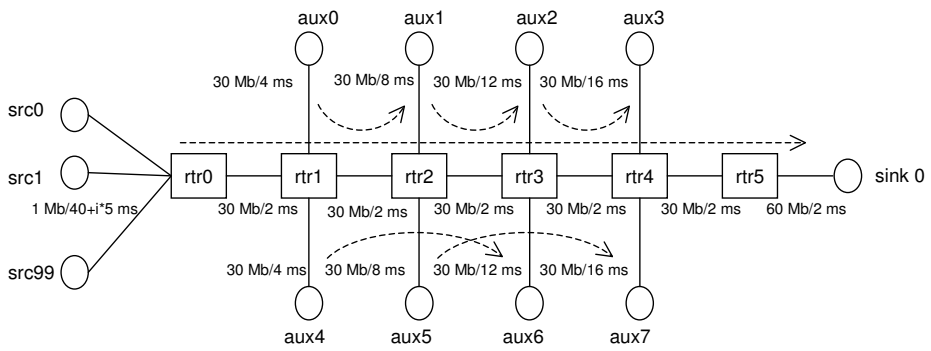

Figure 13: Multiple bottleneck topology.

- Simulations are run for 120 seconds.

From the point of view of the flows of interest (those generated by nodes src0 to src99), the only change is the per-flow assured rate. In this topology, flows share the backbone links with background aggregates. Thus, the total assured rate for each test marking profile (25, 50, 75, 100 and 125%) is assigned in terms of the traffic that traverses the more congested router. As seen in Figure 13, rtr2 is traversed by a maximum of four aggregates: three background aggregates and the one made of the 100 flows of interest. Thus, if total assured rate is X% of 30 Mb/s, each one of the aggregates will be marked for an assured rate of $Y = \frac{X}{4} \cdot 30$ Mb/s. Individual flows from src0 to src99 will then get $Z = \frac{Y}{100}$ Mb/s.

Each background aggregate is made of 50 connections: a combination of 5 FTP, 40 On/Off over TCP and 5 On/Off over UDP. The characteristics of these connections are shown in Table 11.

4.9.2 Results

Figure 14 shows plots of the average queue size for router rtr2, that is, the most congested one. As in section 4.6.1, each plot shows the average queue sizes per AQM. Each line in the plot represents one of the six cases and each value is the average queue size for a particular marking profile. It can be seen that for a given AQM, all curves are very similar. This means that average queue size is more or less the same in the six cases. The reason for this is that, in this topology, traffic from nodes src0–src99 accounts only for a (relatively) small fraction of the overall

Traffic type	Parameter	Value
FTP	Transport protocol	TCP Reno
	Receiver window size	15 packets
On/Off TCP	Transport protocol	TCP Reno
	Receiver window size	15 packets
	Burst and idle time distribution	Pareto
	Mean burst time	350 ms
	Mean idle time	650 ms
	Peak rate	1000 kb/s
	Shape (for the Pareto distribution)	1.5
On/Off UDP	Transport protocol	UDP
	Burst and idle time distribution	Pareto
	Mean burst time	350 ms
	Mean idle time	650 ms
	Peak rate	1000 kb/s
	Shape (for the Pareto distribution)	1.5

Table 11: Parameters of background traffic connections.

load. Thus, changes introduced in this traffic aggregate have a smaller effect on the general performance of the network. However, with RIO, queue size varies sharply for different marking profiles, while A-RIO keeps it near the target for all scenarios. Nevertheless, the values for A-RIO are higher than those observed in the dumbbell topology (Fig. 8). On the average, the difference between the observed average queue sizes and the target value (187.5) is about 24 packets (the maximum and minimum difference observed are 47.6 and 10.6 packets, respectively). For the dumbbell topology the difference is, on the average, -6.4 packets. The higher difference in the complex topology may be attributed to a higher congestion and the presence of non-responsive UDP flows in the background aggregates.

With regard to protection and total throughput, Table 12 shows the overall utilization and percentages of discarded green packets for router rtr2. Here, it can be seen that utilization with A-RIO is generally lower than with RIO, although differences are minor. As explained above, occupation in RIO is considerably higher in profiles with high subscription. Thus, throughput is somewhat higher at the expense of longer queues. Compared to the dumbbell topology, all AQMs discard more green packets. The reason for this, as in the previous section, is the higher level of congestion and the presence of non-adaptive UDP flows. As expected, A-RIO and G-RIO discard more green packets than RIO due to the use of overlapped thresholds.

As of fairness, Table 13 shows the mean per-flow throughput and coefficient of variation for the 100 flows generated by nodes src0 to src99. The data include all

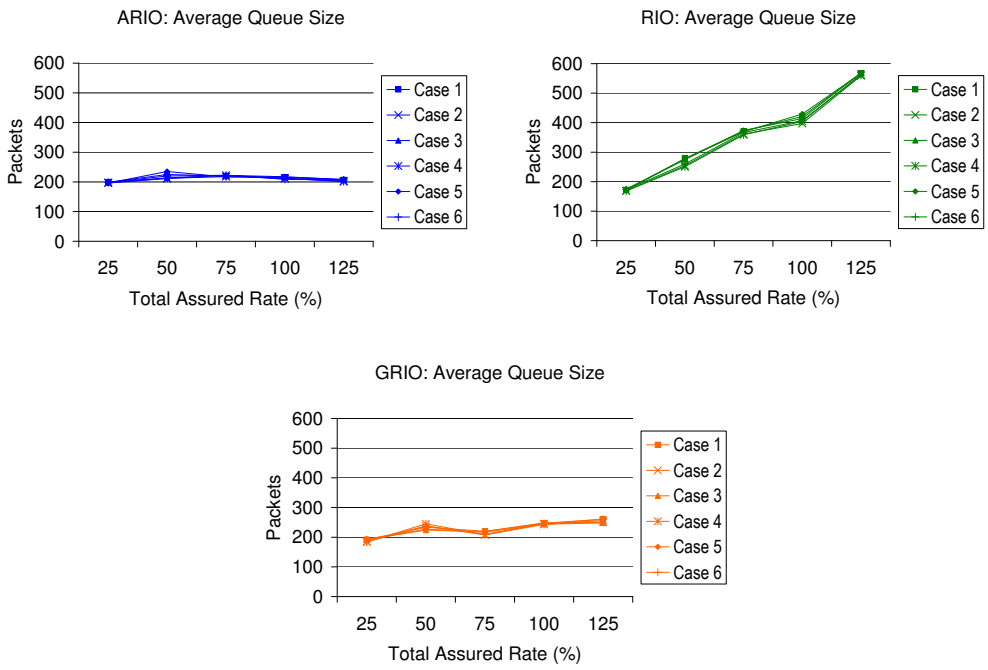

Figure 14: Average queue sizes per AQM, case and marking profile.

cases and marking profiles. No significant differences can be found neither on the mean per-flow throughput nor on the coefficient of variation.

5 Conclusions

In this paper we have presented an adaptive AQM algorithm, A-RIO, suitable for the AF per-hop behaviour of the DiffServ Architecture. A-RIO draws directly from the original RIO proposal and the Adaptive RED (A-RED) algorithm. A-RIO may be helpful in alleviating the tuning problem most AQM algorithms show, as well as in translating a quality-of-service metric into a set of router parameters. Our simulation results suggest that A-RIO could be used as a building block for controlled-delay, AF-based services. A-RIO stabilises queue occupation, without having an adverse effect on other performance parameters like packet discrimination, throughput and

Case	Assured Rate	Utilization (%)			Discarded Green packets (%)		
		RIO	A-RIO	G-RIO	RIO	A-RIO	G-RIO
1	25	93.76	93.30	93.64	0.00	0.00	0.00
	50	93.85	91.56	91.09	0.00	0.09	0.17
	75	94.08	92.62	92.53	0.00	0.98	0.57
	100	94.01	93.20	93.10	1.33	6.71	5.66
	125	93.86	88.14	94.01	5.30	9.27	8.18
2	25	92.58	92.68	92.46	0.00	0.00	0.00
	50	92.34	92.02	91.60	0.00	0.06	0.18
	75	93.06	89.98	89.34	0.00	0.37	0.51
	100	92.97	91.67	92.38	0.89	4.92	4.24
	125	92.91	89.52	92.86	5.07	7.72	8.07
3	25	92.36	90.30	92.39	0.00	0.00	0.00
	50	92.63	91.77	90.89	0.00	0.07	0.14
	75	92.35	92.13	91.13	0.01	0.64	0.71
	100	92.64	90.66	92.44	1.42	6.78	5.42
	125	92.59	87.86	92.71	5.33	9.02	8.54
4	25	92.02	92.10	92.15	0.00	0.00	0.00
	50	92.33	91.07	92.17	0.00	0.05	0.17
	75	92.38	89.84	89.11	0.00	0.33	0.50
	100	92.55	89.91	91.69	0.76	5.30	4.46
	125	92.14	87.91	92.37	4.85	8.13	7.88
5	25	92.47	93.24	93.11	0.00	0.00	0.00
	50	93.22	89.11	90.81	0.00	0.07	0.16
	75	92.79	92.90	91.31	0.01	0.44	0.59
	100	93.15	91.88	92.81	1.50	6.66	4.80
	125	92.96	87.38	93.23	5.53	9.37	8.51
6	25	92.47	90.06	92.27	0.00	0.00	0.00
	50	93.00	90.73	91.27	0.00	0.05	0.21
	75	92.60	90.08	89.64	0.00	0.62	0.44
	100	92.71	92.54	92.02	0.92	4.77	4.69
	125	92.67	89.10	93.08	5.05	7.98	7.77
Mean		92.85	90.84	91.99	1.27	3.01	2.75

Table 12: Percentages of overall link utilization and green packet discard.

fairness in bandwidth sharing. Our results also suggest that the parameter setting rules introduced in [10], combined with the use of overlapped thresholds and the *gentle* option of RED, may be useful in obtaining a more predictable behaviour of non-adaptive RIO, in terms of queuing delay.

There are a few open issues of A-RIO that are worth exploring, for instance: (1) the impact of using a per-precedence averaging weigth $w_q^{(i)}$ on the performance of A-RIO, with respect to the level of differentiation among virtual queues; (2) what happens to fairness in a mixed assured-rate and heterogeneous RTT scenario; (3) a comparison of A-RIO and G-RIO with WRED [1]. Since A-RIO might be used to build services with a loose bound on delay, it may be interesting to evaluate

Case	Assured Rate (%)	RIO		A-RIO		G-RIO	
		Mean Throughput (kbps)	CV	Mean Throughput (kbps)	CV	Mean Throughput (kbps)	CV
1	25	68.80	0.0502	68.49	0.0580	70.04	0.0489
	50	74.10	0.0169	74.92	0.0399	75.74	0.0360
	75	79.36	0.0393	79.87	0.0366	80.47	0.0362
	100	70.31	0.0300	67.77	0.0790	66.55	0.0854
	125	62.62	0.0718	63.35	0.1576	60.49	0.1490
2	25	55.80	0.1664	56.42	0.1516	57.14	0.1508
	50	66.77	0.0976	63.74	0.1243	63.09	0.1338
	75	67.81	0.1240	63.22	0.1628	63.04	0.1604
	100	64.10	0.1035	57.54	0.1775	59.12	0.1398
	125	50.27	0.1851	49.32	0.2567	50.16	0.2379
3	25	69.90	0.0486	69.11	0.0511	69.62	0.0562
	50	73.11	0.0612	72.90	0.1246	75.57	0.0523
	75	79.08	0.0447	79.38	0.0359	79.60	0.0593
	100	71.09	0.0353	67.06	0.1168	67.03	0.1052
	125	61.72	0.0915	63.72	0.1827	62.58	0.1374
4	25	55.46	0.1602	55.67	0.1609	56.80	0.1613
	50	66.73	0.0915	63.76	0.1557	63.54	0.1270
	75	68.63	0.1087	63.84	0.1579	62.77	0.1595
	100	64.32	0.1210	55.65	0.1861	57.95	0.1548
	125	51.39	0.1944	48.68	0.2539	49.78	0.2329
5	25	68.76	0.0565	69.00	0.0557	70.67	0.0510
	50	73.90	0.0158	76.13	0.0379	76.02	0.0409
	75	78.18	0.0633	78.20	0.0426	80.47	0.0605
	100	70.84	0.0410	67.13	0.0912	67.51	0.0975
	125	61.06	0.0782	59.04	0.2232	62.73	0.1687
6	25	59.93	0.1528	59.84	0.1543	60.62	0.1572
	50	69.27	0.0788	65.13	0.1411	65.85	0.1297
	75	71.41	0.0966	67.55	0.1689	66.63	0.1985
	100	66.34	0.1167	62.04	0.1690	61.17	0.1327
	125	53.08	0.1886	52.71	0.2504	53.53	0.2111

Table 13: Fairness measures.

its performance when UDP traffic is used, regarding both delay *and* jitter, as well as the influence of packet size on these quantities. We would also like to look at implementation issues in a real platform. Finally, we intend to work on the analytical modelling of the algorithm.

Acknowledgments

We would like to thank Octavio Medina for his comments on an earlier version of this paper.

References

[1] *Cisco IOS Release 12.1 Quality of Service Solutions Configuration Guide—Congestion Avoidance Overview.* `http://www.cisco.com/`.

[2] T. Bonald, M. May, and J.-C. Bolot. Analytic Evaluation of RED Performance. In *Proceedings of IEEE INFOCOM*, pages 1415–1424, 2000.

[3] B. Braden et al. Recommendations on Queue Management and Congestion Avoidance in the Internet. Internet Standards Track RFC 2309, IETF, April 1998.

[4] M. Christiansen, K. Jaffay, D. Ott, and F. Donelson Smith. Tuning RED for Web Traffic. In *Proceedings of ACM SIGCOMM*, pages 139–150, 2000.

[5] D. Clark and W. Fang. Explicit Allocation of Best-Effort Packet Delivery Service. *IEEE/ACM Transactions on Networking*, 6(4):362–373, August 1998.

[6] W. Feng, D. D. Kandlur, D. Saha, and K. G. Shin. A Self-Configuring RED Gateway. In *Proceedings of IEEE INFOCOM*, pages 1320–1328, 1999.

[7] V. Firoiu and M. Borden. A Study of Active Queue Management for Congestion Control. In *Proceedings of IEEE INFOCOM*, pages 1435–1444, 2000.

[8] S. Floyd. Recommendations on the Use of the *gentle* Variant of RED. `http://www.icir.org/floyd/red/gentle.html`.

[9] S. Floyd. RED: Discussions of Setting Parameters. `http://www.icir.org/floyd/REDparameters.txt`.

[10] S. Floyd, R. Gummadi, and S. Shenker. Adaptive RED: An Algorithm for Increasing the Robustness of RED. Technical Report, to appear, 2001.

[11] S. Floyd and V. Jacobson. Random Early Detection Gateways for Congestion Avoidance. *IEEE/ACM Transactions on Networking*, 1(4):397–413, August 1993.

[12] J. Heinanen, F. Baker, W. Weiss, and J. Wroclawski. Assured Forwarding PHB Group. Internet Standards Track RFC 2597, IETF, June 1999.

[13] J. Heinanen and R. Guerin. A Two Rate Three Color Marker. Informational RFC 2698, IETF, September 1999.

[14] C. V. Hollot, V. Misra, D. F. Towsley, and W. Gong. A Control Theoretic Analysis of RED. In *Proceedings of IEEE INFOCOM*, pages 1510–1519, 2001.

[15] Y. Hori, T. Ikenaga, and Y. Oie. Queue Management of RIO to Achieve High Throughput and Low Delay. *IEICE Transactions on Communications*, E85-B(1):63–69, January 2002.

[16] R. Makkar, I. Lambadaris, J. Hadi Salim, N. Seddigh, B. Nandy, and J. Babiarz. Empirical study of buffer management scheme for DiffServ Assured Forwarding PHB. In *Proceedings of the 9th IEEE International Conference on Computer Communications and Networks*, Las Vegas, October 2000.

[17] N. Malouch and Z. Liu. Performance Analysis of TCP with RIO Routers. Technical Report RR-4469, INRIA, May 2002.

[18] M. May, J. Bolot, C. Diot, and B. Lyles. Reasons not to Deploy RED. In *Proceedings of the 7th International Workshop on Quality of Service (IWQoS'99)*, pages 260–262, London, June 1999.

[19] M. May, J.-C. Bolot, A. Jean-Marie, and C. Diot. Simple Performance Models of Differentiated Services Schemes for the Internet. In *Proceedings of IEEE INFOCOM*, pages 1385–1394, 1999.

[20] O. Medina. *Étude des algorithmes d'attribution de priorités dans un Internet à Différentiation de Services*. Ph.D. dissertation (in French), Université de Rennes 1, March 2001.

[21] O. Medina, J. Orozco, and D. Ros. Bandwidth Sharing under the Assured Forwarding Per-Hop Behavior. Technical Report PI-1478, IRISA, October 2002.

[22] V. Misra, W. Gong, and D. F. Towsley. Fluid-based Analysis of a Network of AQM Routers Supporting TCP Flows with an Application to RED. In *Proceedings of ACM SIGCOMM*, pages 151–160, 2000.

[23] K. Nichols et al. Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers. Internet Standards Track RFC 2474, IETF, December 1998.

[24] W. H. Park, S. Bahk, and H. Kim. A Modified RIO Algorithm that Alleviates the Bandwidth Skew Problem in Internet Differentiated Service. In *Proceedings of IEEE ICC 2000*, volume 1, pages 1599–1603, New Orleans, June 2000.

[25] N. Seddigh, B. Nandy, P. Pieda, J. Hadi Salim, and A. Chapman. An Experimental Study of Assured Services in a DiffServ IP QoS Network. In *Proceedings of SPIE Symposium on Voice, Video and Data Communications*, volume 3529, pages 217–219, December 1998.

[26] T. Ziegler, C. Brandauer, and S. Fdida. A Quantitative Model for the Parameter Setting of RED with TCP Traffic. In *Proceedings of the 9th International Workshop on Quality of Service (IWQoS'01)*, pages 202–216, Karlsruhe, June 2001.

[27] T. Ziegler, S. Fdida, C. Brandauer, and B. Hechenleitner. Stability of RED with Two-Way TCP Traffic. In *Proceedings of IEEE ICCCN 2000*, 2000.